

# Django & DRF로 만드는 블로그 백엔드

The Web framework for perfectionists with deadlines

# 목차

1. Django 간단 소개
2. DRF (Django REST Framework) 소개
3. 요청 처리 흐름
4. 실제 API 만들어보기
5. 개발 단계별 설명
6. Django/DRF의 장점
7. 실제 API 동작 시연
8. 마무리 및 Q&A

# 1. Django 간단 소개

# Django란?

"The Web framework for perfectionists with deadlines"

- 🐍 파이썬 기반의 고수준 웹 프레임워크
- ⚡ Convention over Configuration 철학
- 🚀 빠른 개발과 깔끔한 설계를 추구

# Spring vs Django 아키텍처

## Spring의 전통적인 3계층 구조

Controller → Service → Repository

- **Controller:** 요청 처리 및 응답
- **Service:** 비즈니스 로직 처리
- **Repository:** 데이터 액세스 계층

## Django의 MTV 패턴

Model → Template → View

- **Model:** 데이터 구조와 비즈니스 로직
- **Template:** 사용자에게 보여지는 화면

**View:** 요청을 처리하고 응답을 반환

# Django의 핵심 철학

## DRY (Don't Repeat Yourself)

코드 중복을 최소화하여 유지보수성 향상

## Explicit is better than implicit

명시적인 코드 작성으로 가독성 향상

## Convention over Configuration

설정보다 규칙을 통한 개발 복잡성 감소

## 2. DRF 소개

## DRF (Django REST Framework)란?

-  Django 기반의 강력한 REST API 구축 도구
-  Django의 철학을 그대로 계승
-  API 개발에 특화된 기능들 제공

## API 요청 처리 흐름

```
urls.py → views.py → serializers.py → models.py
```

1. **urls.py**: URL 패턴 매핑
2. **views.py**: 요청 처리 로직
3. **serializers.py**: 데이터 직렬화/역직렬화
4. **models.py**: 데이터 모델 정의

### 3. 요청 처리 흐름

# Django 서버 구조

HTTP Request → WSGI/ASGI → Django → Response

## 핵심 컴포넌트

- **WSGI**: 파이썬 웹 애플리케이션과 웹 서버 간의 표준 인터페이스
- **ASGI**: WSGI의 비동기 버전 (WebSocket, HTTP/2 지원)
- **WAS**: 비즈니스 로직을 처리하는 서버

## 상세 요청 처리 과정

1. HTTP 요청 수신 ( `django/core/servers/basehttp.py` )
2. URL 매칭 (URLconf)
3. 미들웨어 처리
4. View 함수 실행
5. 템플릿 렌더링 (필요시)
6. HTTP 응답 반환

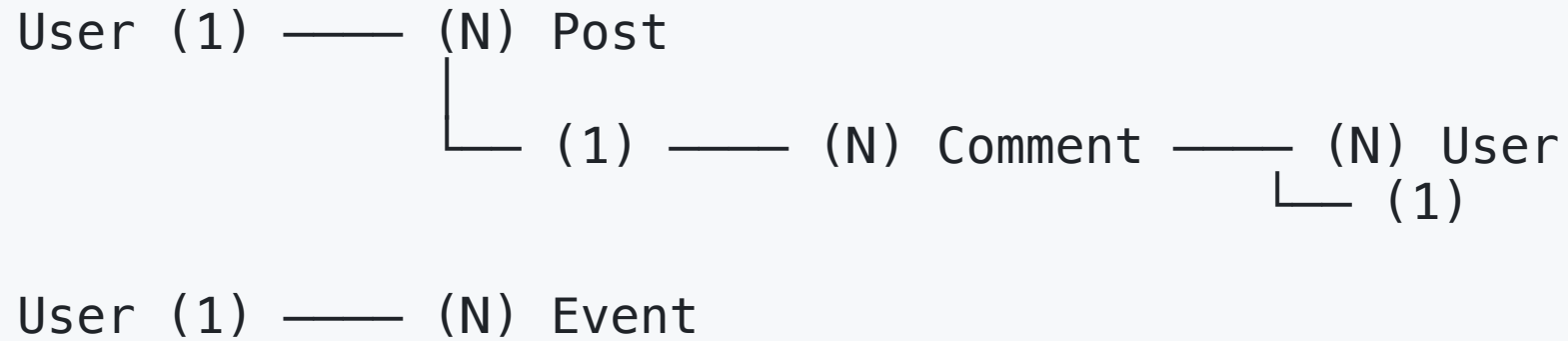
## 4. 실제 API 만들어보기

## 프로젝트 개요

### 간단한 블로그 백엔드 API 구축

- User (사용자)
- Post (게시글)
- Comment (댓글)
- Event (이벤트)

# 데이터 모델 관계



## 주요 기능

- 게시물 관리: CRUD 기능
- 댓글 시스템: 게시물별 댓글 관리
- 이벤트 관리: 행사 정보 관리
- 사용자 인증: 로그인/권한 관리

## 5. 개발 단계별 설명

# 1 프로젝트 구조 세팅

## 기본 설정

- 가상환경 설정
- Django 및 DRF 설치
- 프로젝트 생성 및 앱 구조 설계

## 프로젝트 구조

```
blog_backend/  
├── blog_backend/  # 설정 파일들  
├── blog/          # 메인 앱  
│   ├── models.py  
│   ├── views.py  
│   ├── serializers.py  
│   └── urls.py  
└── manage.py
```

## 2 데이터 모델링

### ERD 설계

- 엔티티 간의 관계 정의
- 필드 타입 선택
- 제약 조건 설정

### 모델 설계 고려사항

- 정규화: 데이터 중복 최소화
- 성능: 인덱싱 고려
- 확장성: 미래 요구사항 고려

### 3 Models 구현

#### Django ORM 활용

- 관계 설정: ForeignKey, ManyToMany
- 메타클래스: 정렬, 테이블명 등 설정
- 메서드: 비즈니스 로직 구현

#### 주요 모델

- **User**: Django 기본 사용자 모델 확장
- **Post**: 게시글 (제목, 내용, 작성자, 날짜)
- **Comment**: 댓글 (내용, 작성자, 게시글)
- **Event**: 이벤트 (제목, 설명, 일시, 장소)

## 4 마이그레이션

### 데이터베이스 스키마 관리

```
python manage.py makemigrations # 마이그레이션 파일 생성  
python manage.py migrate       # 데이터베이스 적용
```

### 마이그레이션의 장점

- 버전 관리: 스키마 변경 이력 추적
- 협업: 팀원 간 스키마 동기화
- 배포: 운영 환경 스키마 업데이트

## 5 Serializers 구현

### 데이터 직렬화/역직렬화

- JSON ↔ Python 객체 변환
- 유효성 검증
- 중첩 시리얼라이저 활용

### 시리얼라이저 종류

- **ModelSerializer**: 모델 기반 자동 생성
- **ListSerializer**: 목록 전용 간소화
- **Custom Serializer**: 특수 로직 구현

## 6 Views 구현

### View 종류

- **APIView**: 클래스 기반 뷰
- **ViewSet**: 자동 CRUD 생성
- **Custom Actions**: 추가 기능 구현

### 권한 관리

- **IsAuthenticated**: 인증된 사용자만
- **IsAuthenticatedOrReadOnly**: 읽기는 모두, 쓰기는 인증된 사용자만
- **Custom Permissions**: 사용자 정의 권한

## 7 URLs 구성

### RESTful API 엔드포인트 설계

```
# Router 활용한 자동 URL 생성
router = DefaultRouter()
router.register(r'posts', PostViewSet)
router.register(r'comments', CommentViewSet)
```

### URL 패턴

- 목록: GET /api/posts/
- 상세: GET /api/posts/{id}/
- 생성: POST /api/posts/
- 수정: PUT /api/posts/{id}/
- 삭제: DELETE /api/posts/{id}/

## 6. Django/DRF의 장점

## ✨ 개발 생산성

### 자동 생성 기능

- Admin 인터페이스: 자동 생성되는 관리자 페이지
- DRFBrowsable API: 자동 생성되는 API 문서
- ORM: 직관적인 데이터베이스 조작

### Built-in 기능

- 인증 시스템: 사용자 관리
- 세션 관리: 상태 유지
- 캐싱: 성능 최적화

## 보안

### 기본 보안 기능

- CSRF 보호: 기본 제공
- SQL Injection 방지: ORM 사용
- XSS 방지: 템플릿 시스템의 자동 이스케이핑

### 추가 보안 기능

- 비밀번호 해싱: 안전한 비밀번호 저장
- 미들웨어: 보안 헤더 자동 추가
- 권한 시스템: 세밀한 접근 제어

## 확장성

### 확장 포인트

- 미들웨어: 요청/응답 처리 확장
- 신호(Signals): 이벤트 기반 처리
- Custom Fields: 사용자 정의 필드 타입

### 생태계

- 서드파티 패키지: 풍부한 생태계
- 플러그인: 다양한 기능 확장
- 커뮤니티: 활발한 개발자 커뮤니티

## 7. 실제 API 동작 시연

# 주요 API 엔드포인트

## 게시글 관리

- `GET /api/posts/` - 게시글 목록 조회
- `POST /api/posts/` - 게시글 생성
- `GET /api/posts/{id}/` - 게시글 상세 조회
- `PUT /api/posts/{id}/` - 게시글 수정
- `DELETE /api/posts/{id}/` - 게시글 삭제

## 댓글 및 이벤트

- `POST /api/posts/{id}/add_comment/` - 댓글 추가
- `GET /api/events/upcoming/` - 다가오는 이벤트 조회

# 개발 도구 활용

## Django Admin

- 데이터 관리: 웹 인터페이스를 통한 데이터 CRUD
- 사용자 관리: 권한 및 그룹 관리
- 자동 생성: 모델 기반 자동 인터페이스

## DRF Browsable API

- API 문서: 자동 생성되는 대화형 문서
- 테스트: 브라우저에서 직접 API 테스트
- 스키마: API 구조 시각화

## 실제 결과물 시연

### ■ API 응답 예시

```
{  
  "id": 1,  
  "title": "Django 시작하기",  
  "content": "Django로 웹 개발을 시작해보자!",  
  "author": {  
    "id": 1,  
    "username": "developer"  
  },  
  "created_at": "2025-07-03T10:00:00Z",  
  "comments_count": 5  
}
```

## 8. 마무리 및 Q&A

## 핵심 포인트 정리

### 1. Django는 빠른 개발이 가능한 프레임워크

- Convention over Configuration
- 풍부한 기본 기능

### 2. DRF는 강력한 API 구축 도구

- 자동 CRUD 생성
- 유연한 시리얼라이저

### 3. 개발 복잡성 감소

- 자동 관리 인터페이스
- 내장 보안 기능

### 4. 확장 가능성

## 다음 단계

### 실제 프로덕션 환경

- 배포: Docker, AWS, Heroku
- 데이터베이스: PostgreSQL, MySQL
- 캐싱: Redis, Memcached

### 고급 기능

- 성능 최적화: 쿼리 최적화, 캐싱
- 테스트: 단위 테스트, 통합 테스트
- 문서화: Swagger, API 문서 자동화

## ? 질문 시간

Django 관련 궁금한 점이 있으신가요?

- 실제 프로젝트 적용 방안
- 다른 프레임워크와의 비교
- 성능 및 확장성 관련
- 학습 로드맵

# 감사합니다!

Django로 함께 멋진 웹 애플리케이션을 만들어보세요! 🚀